

LLM4Teach: LLM-Guided Reinforcement Learning

A Technical Report on PPO with Symbolic Planning, Kickstarting, and Reflection

Phase 3 — Reflection | Environment: SimpleDoorKey | Kaggle GPU Run

Date: June 2026

Abstract

LLM4Teach is a hybrid reinforcement learning framework that combines Proximal Policy Optimization (PPO) with a Large Language Model (LLM) teacher to solve sparse-reward navigation tasks. The LLM acts as a symbolic planner, generating high-level skill plans that are converted into action probability distributions. These distributions shape the PPO agent's learning via a Kickstarting loss — a cross-entropy distillation objective. A persistent plan cache reduces the LLM query cost from 1,500,657 total queries to just 476 actual LLM calls, achieving a 99.97% cache hit rate. A Reflection subsystem uses a second LLM call to analyze completed episodes and store corrective strategy lessons in a bounded memory buffer. These lessons are retrieved and injected into future planning prompts, creating a closed feedback loop between past experience and future planning. The system was trained online while interacting with the environment. Training ran for 2,000 iterations across 20,002 episodes on the MiniGrid SimpleDoorKey environment, achieving a 77.4% overall success rate. This report provides a complete technical walkthrough of every system component.

1. System Overview and Action Space

LLM4Teach integrates three components: a PPO actor-critic neural network (the student), an LLM-based symbolic planner (the teacher), and a reflection memory system. At every step of the environment interaction, the agent chooses one of seven primitive actions.

1.1 Primitive Action Space

The agent operates with a discrete action space of seven primitive actions. These are the only actions the agent can execute in the environment:

Action	Index	Description
Turn Left	0	Rotate the agent 90 degrees counter-clockwise.
Turn Right	1	Rotate the agent 90 degrees clockwise.
Move Forward	2	Move one cell in the direction the agent is currently facing.
Pickup	3	Pick up the object directly in front of the agent (e.g. the key).
Drop	4	Drop the currently held object in the cell directly ahead.
Toggle	5	Interact with the object ahead — opens a door if the agent holds the matching key.
Wait (no-op)	6	No-operation fallback. MiniGrid labels this 'done', but in SimpleDoorKey the episode ends automatically — this signal is never needed. The teacher emits it only as a last resort when no valid skill action can be computed.

The LLM teacher and the PPO student both reason about these seven actions. The teacher's skill system converts a high-level plan (such as 'go to key') into a probability distribution over these seven actions at every step. PPO then samples from its own distribution to select the action executed in the environment.

1.2 High-Level System Flow

Each training step follows this order:

- **Observation:** The agent receives a partial view of the grid (3×3 tiles ahead).
- **Failure Detection:** The failure detector checks whether the agent is stuck, oscillating, or repeatedly failing interactions.
- **Cache Lookup:** The symbolic observation is encoded and checked against the plan cache.
- **Plan Retrieval or Generation:** If a cached plan exists, it is reused. Otherwise the LLM is queried to generate a new plan.
- **Skill Execution:** The active skill converts the current plan step into action probabilities.
- **PPO Action:** The student policy samples an action from its own distribution and executes it.
- **Buffer Storage:** The observation, action, reward, and teacher probabilities are stored for PPO training.

2. Environment Description

All experiments in this report use a single environment: **MiniGrid SimpleDoorKey**. The task is to find a key placed randomly in the room, pick it up, navigate to a locked door, and toggle the door open to reach the goal. This requires the agent to perform a structured sequence of sub-tasks in the correct order.

2.1 Environment Specification

Property	Value	Notes
Environment name	MiniGrid-SimpleDoorKey-Min5-Max10-View3	Gymnasium ID used to create the environment.
Room size	5 to 10 tiles (randomized per episode)	A new room size is sampled at the start of each episode within this range. Not fixed at 5x5.
Agent view	3x3 tiles directly ahead (partial observability)	The agent cannot see the full room at once.
Maximum episode length	150 steps	If the agent does not complete the task within 150 steps, the episode terminates with zero reward.
Reward on success	$1 - 0.9 \times (\text{steps} / 150)$	Higher reward for faster completion. Maximum reward is 1.0 (immediate success).
Reward on failure	0.0	Episode timeout or any failure results in zero reward.
Task sequence	Find key → Pick up key → Navigate to door → Toggle door → Reach goal	The agent must follow this order. Toggling the door without the key does nothing.
Key and door positions	Randomized per episode	Positions are not fixed. The agent must explore to find them.

2.2 Why This Environment?

SimpleDoorKey requires the agent to: (1) explore when objects are not visible, (2) navigate to specific objects, (3) pick up items, and (4) use items to interact with the environment. This multi-step structure makes it a suitable benchmark for testing whether an LLM teacher can provide meaningful high-level guidance to a PPO agent. The partial observability and random placement ensure the agent cannot memorize solutions.

3. Observation Encoding: RL2LLM and RL2LLM_rich

The raw MiniGrid observation is a pixel grid — not directly usable by an LLM. Two separate text encoders convert the observation into different text representations for different purposes.

3.1 RL2LLM — Compact Symbolic Encoding (Cache Key)

RL2LLM is called at every single step. It scans the agent's 3×3 view and produces a short text string listing: (1) what objects are visible in the view, and (2) what the agent is currently holding. It ignores exact positions and distances — only object presence matters.

The output format is always: **Agent sees <object1>, <object2>, holds <item>.** If nothing is visible the placeholder <nothing> is used. This string is used directly as the dictionary key for the plan cache — the cache lookup is an exact string match.

Because SimpleDoorKey contains only two objects (one key and one door), and the agent either holds the key or does not, the total number of distinct strings that can ever be produced is very small. In the entire 1,500,657-step training run only 6 unique strings appeared.

3.2 The 6 Unique Cache Key Strings

These are the exact strings produced by RL2LLM — they are the literal plan cache keys:

Exact RL2LLM Output String	Situation
"Agent sees <nothing>, holds <nothing>."	Nothing in view and not holding anything. Agent must explore.
"Agent sees <key>, holds <nothing>."	Key is visible in the 3x3 view. Agent has not yet picked it up.
"Agent sees <door>, holds <nothing>."	Door is visible but agent has no key. Cannot open door yet.
"Agent sees <nothing>, holds <key>."	Agent is holding the key but neither key nor door is visible. Must navigate to door.
"Agent sees <door>, holds <key>."	Door is visible and agent is holding the key. Ready to toggle.
"Agent sees <key>, <door>, holds <nothing>."	Both key and door are visible simultaneously but agent holds nothing.

These strings are produced by mediator.py RL2LLM() with color_info=False (SimpleDoorKey_Mediator overrides the base class to disable color). The output depends only on what objects appear in the agent's current 3x3 view — not on room size, agent direction, or exact object positions.

3.3 RL2LLM_rich — Relational Encoding (LLM Prompt Input)

RL2LLM_rich is called only when the plan cache misses — that is, only when the LLM actually needs to be queried. It produces a much longer description that includes the direction and approximate distance to visible objects, what the agent is holding, and recent interaction history. This richer description gives the LLM enough spatial context to generate a meaningful plan. It is never used for cache key matching.

Property	RL2LLM (compact)	RL2LLM_rich (relational)
Called when	Every step (1,500,657 times)	Cache miss only (476 times in training)

Property	RL2LLM (compact)	RL2LLM_rich (relational)
Output format	'Agent sees <X>, holds <Y>.' — 6 possible strings	Full relational description — 200+ unique strings across training
Used for	Plan cache lookup key (exact string match)	LLM prompt input only
Includes position	No — only object presence in 3x3 view	Yes — direction, distance, relative location to objects
Example output	"Agent sees <door>, holds <key>."	"Agent faces north. Door is 1 tile ahead. Agent holds key."

4. Failure Detection

Failure detection runs at every step, before any planning or cache lookup occurs. Its purpose is to identify when the agent is stuck in an unproductive behavioral pattern and trigger a replanning intervention. The system was trained online while interacting with the environment — failure detection therefore operates in real time during training.

4.1 Three Failure Patterns

The failure detector monitors three distinct types of unproductive behavior:

Failure Type	What It Means	Detection Condition	Progress Guard
Stuck	The agent visits very few distinct positions. It is physically trapped or moving in a tiny area.	Fewer than 3 unique grid positions visited in the most recent 20 steps.	Only fires if the agent is also NOT making forward progress (fewer than 3 distinct positions in last 30 steps).
Oscillation	The agent alternates back and forth between Left and Right actions repeatedly. This is a sign of indecision or a looping plan.	More than 75% of the last 16 actions consist of alternating Left-Right pairs.	Only fires if the agent is also NOT making forward progress.
Failed Interactions	The agent repeatedly attempts an action that cannot succeed. For example, trying to pick up an object when none is adjacent, or toggling a door without holding the key. After five such failures the detector fires regardless of position history.	Five or more consecutive failed interaction attempts.	No progress guard — fires unconditionally after five failures.

4.2 What Happens When Failure is Detected

When a failure pattern is confirmed, the following sequence of steps occurs conceptually:

- **Failure reported:** The failure is reported to the planner. A failure counter for the current symbolic state is incremented.
- **Cache invalidation (conditional):** If the same symbolic state has failed five or more times, its cached plan is deleted. This forces the LLM to generate a new plan the next time this state is encountered.
- **Failure context created:** A short text description of the failure type is created (e.g., 'previously stuck at this state').
- **Context attached to future plans:** This failure description is attached to the next LLM planning call, so the LLM is aware of what went wrong.
- **Detector reset:** The failure detector's internal state is cleared.

- **Cooldown period:** Interventions are suppressed for the next 30 steps, preventing repeated intervention storms.

Actual Implementation Code — Intervention Trigger

```
# Failure detected at step t:
# 1. Report failure to planner
self.planner.report_failure(obs_text)
#     Inside report_failure(): failure_count[obs_text] += 1
#     If failure_count[obs_text] >= 5: del plans_dict[obs_text]
# 2. Build failure context string
if reason == "stuck":
    failure_ctx = "Previously stuck at this state. Try a different approach."
elif reason == "oscillation":
    failure_ctx = "Previously oscillating at this state. Try a different approach."
elif reason == "failed_interaction":
    failure_ctx = "Previously failed_interaction. Try a different approach."
# 3. Attach context to mediator via method call (not direct attribute)
mediator.set_failure_context(failure_ctx)
# 4. Reset detector and start 30-step cooldown
self._detector.reset()
self._cooldown_remaining = 30    # no new interventions for 30 steps
```

The failure context is a one-time hint injected into the next LLM prompt on cache miss. It is NOT stored in the plan cache itself.

5. Plan Generation and Caching

Calling an LLM at every timestep would require 1,500,657 LLM queries during training — completely impractical. The plan cache solves this by storing the LLM's answer for each symbolic state and reusing it whenever the same state recurs.

Since RL2LLM produces only 6 unique symbolic strings, the cache contains at most 6 entries. The same cached plan is therefore reused for the vast majority of steps. In the training run, 1,500,181 out of 1,500,657 queries were served from cache — a 99.97% hit rate. The LLM was actually called only 476 times.

5.1 Why Were There 476 LLM Calls?

LLM calls occur in two situations:

- **First encounter:** When a symbolic state appears for the first time, no cached plan exists yet. The LLM is called to generate the initial plan for that state.
- **After cache invalidation:** When the failure detector reports five consecutive failures for a state, the cached plan is deleted. The next time that state appears, the cache misses again and the LLM is called to generate a new plan — this time with the failure context injected into the prompt.

With only 6 possible states and 470 cache invalidations during training, the 476 total LLM calls are accounted for by 6 initial plans plus 470 regenerations after failures. Detailed failure handling is described in Section 4.

5.2 Cache Lookup Decision Logic

Situation	Cache Action	LLM Called?
New symbolic state never seen before	MISS — no entry yet	YES
Familiar state with fewer than 5 failures	HIT — plan returned	NO
Familiar state on its 5th failure	HIT (last use) then DELETED immediately after	NO (but next encounter will be a MISS)
State after cache deletion	MISS — entry deleted	YES — with failure context in prompt
Plan timeout (80 steps on same plan)	SOFT RESET — timer resets, cache NOT deleted	NO — same plan continues

5.3 From LLM Output to Plan Distribution

When the LLM is called, it outputs a high-level plan string such as 'go to key, pick up key, open door'. This raw text is processed through two stages before being stored in the cache:

Stage 1 — Symbolic Grammar Validation

The plan string is split by commas into tokens. Each token is matched against the set of known skill verbs: explore, go to, pick up, drop, open. Unknown verbs are replaced with 'explore'. If no valid tokens remain at all, the entire plan is replaced with 'explore' as a safe fallback.

Stage 2 — Token-by-Token Repair

After grammar validation, the plan is checked against the current symbolic state (parsed from the RL2LLM cache key string). Each token is evaluated and repaired independently using five rules:

Rule	Condition	What Happens
R1	Plan says 'go to key' but key is NOT visible in current view and agent is not holding it.	Token is dropped. An 'explore' prefix is added to the final plan so the agent first finds the key.
R2	Plan says 'pick up key' but agent is already holding the key.	Token is silently removed — picking up again is impossible and redundant.
R3	Plan says 'go to door' but door is NOT visible in current view.	Token is dropped. An 'explore' prefix is added so the agent first finds the door.
R4	Plan says 'open door' but agent is NOT holding the key.	If key is visible: 'go to key, pick up key' is injected before the open token. If key is not visible: token is dropped and 'explore' prefix is added.
R5	Current state is 'Agent sees <nothing>, holds <nothing>' — nothing visible and nothing held.	Entire plan is discarded immediately. Replaced with 'explore' regardless of what the LLM said.
R6	After all repairs, consecutive identical tokens exist (e.g. 'explore, explore').	Duplicates are collapsed — only one instance is kept.

The repair runs on the RL2LLM string (compact cache key), not the rich observation. It fires immediately after the LLM responds, before the plan is stored in the cache. This means the cached plan is always consistent with the state that triggered the LLM call.

Stage 3 — Diversification

The repaired plan string is then split into a primary plan and a backup 'explore' component. The weights depend on how many times this symbolic state has previously failed:

Actual Implementation Code — Plan Diversification

```
# After LLM output is validated and repaired:
def _diversify_plan(plan, failures):
    if plan == "explore":
        return ["explore"], [1.0]
    if failures >= 3:
        return [plan, "explore"], [0.40, 0.60]    # heavy explore after many failures
    if failures >= 1:
        return [plan, "explore"], [0.70, 0.30]    # some explore after first failure
    return [plan, "explore"], [0.85, 0.15]        # default: 85% primary, 15% explore

# Result stored in cache:
plans_dict[obs_text] = ([plan, "explore"], [0.85, 0.15])
```

5.4 Actual LLM Planner Prompts

The following are the exact prompts used in the Kaggle training run. These are copied verbatim from the source code.

Planner System Prompt (sent to LLM on every call)

Source: utils/qwen_llm.py — MINIGRID_SYSTEM_PROMPT

You are a strategic MiniGrid planning agent.

Output ONLY a compact symbolic action plan – no explanations, no markdown.

ALLOWED ACTIONS:

```
explore
go to <key> | go to <door> | go to <goal>
pick up <key>
open <door>
drop <key>
```

CRITICAL VISIBILITY RULE (most important):

You can ONLY use 'go to <X>' or 'pick up <X>' for objects that appear in 'Agent sees ...'.

If an object is NOT listed in the current observation -> you CANNOT navigate to it.

If objects are NOT visible -> use 'explore' first to find them.

TASK CONSTRAINTS:

1. LOCKED door requires matching key held -- CANNOT open without key.
2. Door not held key -> find key first. Door visible + key held -> go open it.
3. Nothing visible -> explore.
4. Already holding key -> NEVER generate 'pick up <key>' again.
5. Object not in 'Agent sees ...' -> NEVER generate 'go to <object>'.
6. Previous strategy failed -> change approach, do NOT repeat same plan.
7. Prefer shortest valid plan -- no redundant steps.

STATE -> CORRECT PLAN EXAMPLES:

```
'sees <nothing>, holds <nothing>'      -> explore
'sees <door>,      holds <nothing>'     -> explore
'sees <key>,      holds <nothing>'      -> go to <key>, pick up <key>
'sees <key>, <door>, holds <nothing>'    -> go to <key>, pick up <key>, go to <door>, open <door>
'sees <door>,      holds <key>'         -> go to <door>, open <door>
'sees <nothing>, holds <key>'           -> explore
```

OUTPUT FORMAT: comma-separated symbolic actions only. No text.

GOOD: go to <key>, pick up <key>, go to <door>, open <door>

BAD: go to <key> (when key not in 'Agent sees')

BAD: pick up <key> (when already holding key)

BAD: open <door> (when not holding key)

BAD: turn left, move forward

Planner User Prompt (built per LLM call)

The user-side prompt is assembled by `_build_prompt_with_reflection()` in `planner.py`. It has three parts, assembled in order:

- **Past experience (optional):** Up to 5 reflections retrieved from memory, injected only on cache miss. Present in 475 out of 476 calls.
- **Current state:** The RL2LLM_rich relational description of the current observation (not the compact cache key).

- **Task constraints (optional):** Short inferred implication of the current state, e.g. 'You must pick up the key before you can open the door.'

Example Planner User Prompt — cache miss with memory

Past experience:

[SUCCESS] Agent picked up key immediately when visible, then navigated directly to door. Next time: go to key as soon as it appears in view.

[KEY_FIRST] Agent toggled door 8 times without key. Next time: pick up key BEFORE approaching the door.

[EXPLORE_FRONTIER] Agent looped in seen area. Next time: turn toward open space to reach unseen areas.

Current state:

Agent faces east. Key is visible 2 tiles ahead. No door visible.

Agent holds nothing.

Task constraints:

Key is visible -- navigate to it and pick it up.

If no memory is available (first call) or the token budget is exceeded, the 'Past experience' block is omitted. The LLM then responds with a comma-separated plan string such as: 'go to , pick up '

Correction Prompt (sent on parse failure)

If the LLM's first response cannot be parsed into a valid plan, a second call is made with this correction prompt appended:

Your previous response was not a valid symbolic plan.

Remember the key constraint: a locked door requires the key held first.

Output ONLY comma-separated symbolic actions. Example:

go to <key>, pick up <key>, go to <door>, open <door>

parser_rejections = 0 in the Kaggle run — the correction prompt was never needed. The LLM consistently produced valid plans on the first attempt.

6. Skill Execution

The LLM planner outputs high-level skill names such as 'explore', 'go to key', or 'pick up key'. Each skill is a specialized navigation module that translates these high-level instructions into a probability distribution over the seven primitive actions at every step. The PPO agent receives this distribution through the Kickstarting objective (described in Section 9).

6.1 Available Skills

Explore Skill

The Explore skill is used when the planner does not know where the key or door is located. It navigates the agent toward unseen areas of the grid, encouraging broad coverage of the room. The skill tracks which cells have been visited and steers toward frontiers — boundary areas between explored and unexplored regions. If the agent makes no new discoveries for 25 consecutive steps, the skill terminates and the planner is consulted again.

GoTo Skill

The GoTo skill navigates the agent toward a specific target object — for example, the key, the door, or the goal location. It uses a pathfinding algorithm to compute the shortest path to the target given what the agent can currently see. At each step, it recommends the primitive action most likely to move the agent closer to the target: typically Move Forward, Turn Left, or Turn Right.

Pickup Skill

The Pickup skill positions the agent adjacent to and facing the key, then executes the Pickup action. If the agent is not yet aligned with the key, it first produces turning or movement actions to correct its position.

Toggle Skill

The Toggle skill positions the agent in front of the door and executes the Toggle action. This is only meaningful when the agent is already holding the key. If the agent attempts to toggle without the key, the interaction fails and the failure detector increments the failed-interaction counter.

6.2 From Skills to Action Probabilities

Each skill returns a one-hot action vector — a seven-element array with a 1.0 at the recommended action and 0.0 elsewhere. Because the plan distribution includes a primary plan and a backup 'explore' (with weights such as 0.85 and 0.15), the teacher policy computes a weighted sum of the two skill outputs:

$$p_{teacher} = 0.85 \cdot e_{primary} + 0.15 \cdot e_{explore}$$

The result is a soft probability distribution over the seven primitive actions. For example, if the primary plan recommends Move Forward (index 2) and the explore skill recommends Turn Left (index 0), the teacher distribution is:

[0.15, 0.00, 0.85, 0.00, 0.00, 0.00, 0.00]

Left=15%, Forward=85%, all others=0%. This vector is stored in the experience buffer at every step alongside the PPO agent's own action. It is used exclusively for computing the Kickstarting loss during training.

Actual Implementation Code — Teacher Action Computation

```
# For each plan in the diversified plan list:
teacher_action = zeros(7) # 7 primitive actions
for skill_name, prob in zip(plan_list, prob_list):
    one_hot = get_action(skill_name, current_obs) # returns [0,0,1,0,0,0,0] etc.
    teacher_action += one_hot * prob
# teacher_action is stored in buffer every step:
buffer.store(obs, student_action, reward, value, log_prob, teacher_action)
# student_action = what PPO chose (goes to environment)
# teacher_action = soft distribution (used only for KS loss)
```

7. What Happens at Each Training Step

At every single step during training, the following sequence executes. The system was trained online — the agent interacts with the live environment and all components operate in real time.

7.1 The Cached-Plan Path (99.97% of steps)

In the vast majority of steps, a plan is already cached for the current state. No LLM call occurs. The sequence is:

- The agent receives its observation from the environment.
- The failure detector checks whether the agent is stuck, oscillating, or repeatedly failing.
- The compact symbolic observation (RL2LLM) is computed and used as the cache key.
- The cached plan and its probability weights are retrieved instantly.
- The active skill converts the plan into a teacher action probability distribution.
- The student PPO policy runs a forward pass to produce its own action distribution.
- The student samples an action from its distribution and executes it in the environment.
- The observation, student action, reward, student value estimate, and teacher distribution are stored in the experience buffer.

7.2 The LLM-Planning Path (0.03% of steps — 476 total)

On a cache miss, the LLM must be queried. This path is identical to the above except for steps 3 and 4:

- The compact symbolic observation (RL2LLM) is computed and the cache is checked — no entry found.
- The rich relational observation (RL2LLM_rich) is computed. This includes directional and distance information about visible objects.
- Relevant reflections are retrieved from the reflection memory buffer (up to 5 lessons).
- The LLM is queried with: the rich observation, the retrieved reflections, and optionally a failure context.
- The LLM output is validated, repaired if needed, and diversified into a (plan, probability) distribution.
- The new plan is stored in the cache for all future steps with the same symbolic state.
- Execution continues as in the cached path.

Key insight: Reflection retrieval occurs ONLY on a cache miss, when a new plan must be generated. On cache hits (99.97% of steps), no reflection retrieval occurs. This is one of the primary efficiency gains of the system.

8. What Happens Each Training Iteration

Training proceeds in iterations. Each iteration consists of collecting experience from the environment, updating the PPO policy, and logging metrics. In total, 2,000 iterations were run during the Kaggle experiment.

8.1 Episode and Iteration Counts

The relationship between episodes and iterations is as follows:

- **Episodes per iteration:** 10 episodes are collected at the start of each iteration. Each episode runs until the agent succeeds or 150 steps elapse.
- **Minimum buffer requirement:** After collecting 10 episodes, the system checks whether the experience buffer contains enough transitions for PPO training. The minimum requirement is 256 transitions. If this threshold is not met, additional episodes are collected until it is.
- **Why 20,002 instead of 20,000:** With 2,000 iterations and 10 episodes each, exactly 20,000 episodes would be expected. The extra 2 episodes were collected as buffer top-up during iterations where 10 episodes produced fewer than 256 transitions — typically very early in training when episodes terminate quickly.
- **Total timesteps:** 1,500,657 environment steps across all 20,002 episodes.

8.2 Conceptual Iteration Structure

Each iteration proceeds through the following stages:

- **Collect experience:** Run 10 episodes online. At each step, store the observation, action, reward, and teacher distribution in the buffer.
- **Ensure sufficient samples:** If fewer than 256 transitions have been collected, run additional episodes until the threshold is met.
- **Update PPO:** Run 8 epochs of PPO updates over the collected buffer using mini-batches of 128 transitions. For each mini-batch, compute five losses and update the network weights.
- **Decay teacher influence:** Update the kickstarting coefficient based on the recent success rate.
- **Log metrics:** Record success rate, average reward, episode length, loss values, and system statistics.
- **Save checkpoint:** Every 200 iterations, save the current model weights to disk.

Actual Implementation Code — Iteration Loop

```
for iteration in range(2000):
    buffer.clear()
    # Collect base 10 episodes
    for _ in range(10):
        game.collect() # runs one full episode, stores in buffer
    # Top-up if insufficient transitions
    while len(buffer) < 256:
        game.collect() # extra episode (explains 20,002 total episodes)
    # PPO update: 8 epochs, batch size 128
    for epoch in range(8):
        for minibatch in buffer.sample(128):
            compute_losses -> backward() -> optimizer.step()
    # Decay kickstarting coefficient
    update_kickstarting_coef(recent_success_rate)
```

```
# Checkpoint every 200 iterations
if iteration % 200 == 0:
    save_checkpoint(model)
```

9. Loss Functions and Mathematics

PPO training uses five loss components. The total loss is a weighted combination. The Kickstarting loss is the mathematical bridge between the LLM teacher and the PPO student.

9.1 Advantage Estimation

Advantages measure how much better an action was compared to what was expected. They are computed using Monte Carlo returns:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^n G_{t+n}$$

$$A_t = G_t - V(s_t)$$

Advantages are then batch-normalised: $A = (A - \text{mean}(A)) / (\text{std}(A) + 1e-5)$. The epsilon is 1e-5 (not 1e-8 — confirmed from buffer code).

9.2 Clipped Policy (PPO) Loss

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

$$\mathcal{L}_{CLIP} = -E[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 0.8, 1.2)A_t)]$$

The clipping range is [0.8, 1.2] (clip_eps = 0.2). The clipped objective prevents excessively large policy updates in a single step.

9.3 Value Loss

$$\mathcal{L}_{VF} = E[\max((V(s_t) - G_t)^2, (V_{clip}(s_t) - G_t)^2)]$$

Value clipping uses the same epsilon as the policy clip. Coefficient c_v = 0.5.

9.4 Entropy Loss

$$H(\pi) = -\sum_a \pi(a | s) \cdot \log \pi(a | s)$$

Entropy is maximised (subtracted from the loss) to encourage exploration. A dynamic coefficient is applied: if entropy falls below 0.5 nats, the coefficient doubles. If it falls below 0.2 nats, the coefficient triples. This prevents irreversible policy collapse.

$$\alpha_{eff} = 3\alpha \quad \text{if } H < 0.2 \quad (\text{entropy critical})$$

$$\alpha_{eff} = 2\alpha \quad \text{if } H < 0.5 \quad (\text{entropy low})$$

$$\alpha_{eff} = \alpha \quad \text{otherwise}$$

9.5 Kickstarting (KS) Loss — LLM Knowledge Transfer

The Kickstarting loss is a cross-entropy distillation objective. It measures the divergence between the teacher's action distribution and the student's policy:

$$\mathcal{L}_{KS} = -\sum_a \pi_T(a | s) \cdot \log \pi_S(a | s)$$

where π_T is the teacher's action distribution (from skill execution) and π_S is the student PPO policy's softmax output. This loss is applied only while $\text{iteration} < \text{iter_with_ks} = 3000$. Since training ran for 2,000 iterations, the KS loss was active for the entire run.

Actual Implementation Code — KS Loss

```
# Correct cross-entropy kickstarting formula (algorithms/ppo.py line 105):
kickstarting_loss = -(teacher_prob_batch
                      * F.log_softmax(pdf.logits, dim=-1)
                      ).sum(dim=-1).mean()
```

9.6 Kickstarting Coefficient Decay

The KS coefficient controls how strongly the teacher influences learning. It starts at 1.0 and decays over time based on the agent's recent success rate. The teacher influence coefficient decreases during training and eventually reaches a constant floor value of 0.15. Teacher guidance therefore remains active throughout training, although its effect becomes much smaller after convergence.

$$\Delta = 3\alpha_d \quad \text{if } sr \geq 0.30 \quad (\text{fast decay})$$

$$\Delta = 2\alpha_d \quad \text{if } sr \geq 0.10 \quad (\text{medium decay})$$

$$\Delta = \alpha_d \quad \text{otherwise} \quad (\text{slow decay})$$

$$ks = \max(ks_{min}, ks - \Delta)$$

Parameter values: $\alpha_{descent} = 0.005$, $ks_{minimum} = 0.15$, $ks_{initial} = 0.995$ (confirmed from `experiment_config.py`, which overrides the PPO constructor defaults of 0.02 and 0.05).

Observed decay trajectory from training data: During the first 70 iterations, the success rate was 0% — no successful episodes occurred. Only the base decay rate applied: 0.005 per iteration. This reduced the coefficient from 0.995 to approximately 0.645 over 70 iterations ($70 \times 0.005 = 0.350$ total reduction). From iteration 70 onward, the first successes appeared. With success rate reaching 0.1 to 0.3, the decay accelerated to 2× or 3× speed. The floor of 0.15 was reached at iteration 122 — confirmed from training logs. From iteration 122 onward, the coefficient remained fixed at 0.15 for the remaining 1,878 iterations (94% of the total training run).

9.7 Total Combined Loss

$$\mathcal{L} = \mathcal{L}_{CLIP} + c_v \mathcal{L}_{VF} - \alpha_{eff} H(\pi) + ks(t) \cdot \mathcal{L}_{KS}$$

$c_v = 0.5$ (value loss coefficient). The KS term is only added when the current iteration is less than $iter_with_ks = 3000$. Since training used 2,000 iterations, the KS term was included in every update.

9.8 Reward Function

$$R = 1 - 0.9 \times \frac{steps}{max_steps} \quad (\text{success, } max_steps = 150)$$

$$R = 0.0 \quad (\text{timeout or failure})$$

This reward structure incentivizes the agent to complete the task as quickly as possible. An episode completed in 1 step earns reward 0.994. An episode completed in 150 steps (the maximum) earns reward 0.10. An episode that times out earns 0.

10. Reflection System

After each episode ends, the system may generate a reflection — a short strategic lesson about what went wrong or what worked. These lessons are stored in a bounded memory buffer and injected into future LLM planning prompts, closing the feedback loop between past experience and future planning.

10.1 When Reflection is Triggered

Reflection is NOT triggered after every episode. It uses a rate-limiter to avoid redundant reflections from repeated identical failures:

- **Successful episodes:** Reflection is ALWAYS triggered. Every successful episode produces a reflection regardless of how recently the last reflection occurred.
- **Failed episodes:** Reflection is triggered only if at least 3 episodes have passed since the last reflection. This prevents the system from generating dozens of near-identical reflections during a long stuck period.

10.2 What Is in the Reflection Prompt

The reflection prompt sent to the LLM contains the following information, derived entirely from the symbolic episode trajectory — no raw grid data is included:

Field	Content	Example
Episode result	SUCCESS or FAILURE	'Episode result: FAILURE'
Failure type	Classified failure label	'Failure type: toggle_without_key'
Episode length	Number of steps taken	'Episode length: 142 steps'
Total reward	Cumulative reward received	'Total reward: 0.00'
Observations seen	Top 5 most frequent symbolic states (RL2LLM output)	'holds nothing, key visible (x89)'
Plans executed	Top 4 most frequent plan strings	'go to key, pick up key (x98)'
Failure diagnostic hint	Optional note added when specific failure types are detected	'Note: agent tried to open door without the key'
Skill failure signals	Signals from skills when interactions failed	'toggle_no_key (x12)'
Intervention events	List of failure interventions that fired during the episode	'stuck intervention (x2)'

The reflection prompt does NOT include: agent grid position, key exact position, door exact position, or number of unexplored cells. All content comes from symbolic RL2LLM observations and plan strings only.

10.3 Actual Reflection Prompts

Reflection System Prompt (sent verbatim to reflector LLM)

Source: `memory/reflection.py` — `REFLECTION_SYSTEM_PROMPT`

You are an execution diagnostician for a MiniGrid RL agent.

Your job: identify the SPECIFIC execution mistake and give ONE corrective strategy.

Write in EXACTLY 2 sentences. No more.

Sentence 1 -- What went wrong (be specific about the failure mechanism):

GOOD: 'Agent approached the door and toggled it 8 times without holding the key.'

GOOD: 'Agent explored the same corridor repeatedly without reaching unseen areas.'

GOOD: 'Agent picked up the key but failed to navigate to the door before timeout.'

BAD: 'Agent failed this episode.' <- too vague

BAD: 'The task was not completed.' <- no diagnosis

Sentence 2 -- ONE concrete corrective strategy:

GOOD: 'Next time: pick up the key BEFORE approaching the door.'

GOOD: 'Next time: after picking up the key, navigate directly to the door.'

GOOD: 'Next time: explore unseen areas by turning toward open space.'

BAD: 'Try a different approach.' <- too vague

STRICT RULES:

1. NO coordinates, NO grid positions
2. NO speculation about unseen objects
3. NO markdown, no bullets, no headers
4. ONLY reference objects actually seen: key, door, lava, wall
5. Focus on EXECUTION mechanics, not high-level outcomes
6. If episode succeeded: describe WHY it worked in 1 sentence only

Reflection User Prompt — built from episode trajectory (`EpisodeTrajectory.to_prompt()`)

This is an example of the user-side prompt sent to the reflector after a failed episode of type 'toggle_without_key'. The content is assembled from the symbolic trajectory only — no raw observations, no grid coordinates.

Episode result: FAILURE

Failure type: toggle_without_key

Episode length: 142 steps

Total reward: 0.00

Observations seen (most frequent first):

- Agent sees <door>, holds <nothing>. (repeated x89)
- Agent sees <nothing>, holds <nothing>. (repeated x43)
- Agent sees <key>, holds <nothing>. (repeated x10)

Plans executed (most frequent first):

- go to <door>, open <door> (x98)
- explore (x31)
- go to <key>, pick up <key> (x13)

Note: The agent tried to open the door without picking up the key first.

Skill failure signals:

- toggle_attempt_failed (x34)
- key_picked_attempt (x12)

Intervention events: 3 total

- failed_interaction intervention
- stuck intervention

In exactly 2 sentences:

Sentence 1: What specific execution mistake caused this outcome?

Sentence 2: ONE concrete corrective action for next time.

ONLY describe what actually happened. Do NOT speculate.

A typical LLM response to the above prompt: 'Agent repeatedly attempted to open the door 34 times without first picking up the key, causing all toggle attempts to fail. Next time: pick up the key before navigating to the door.'

10.4 The Reflection System Prompt Rules

The reflector LLM is instructed to respond in exactly two sentences:

- **Sentence 1:** What specific execution mistake caused this outcome? (e.g., 'Agent approached the door and toggled it 8 times without holding the key.')
- **Sentence 2:** ONE concrete corrective strategy for next time. (e.g., 'Next time: pick up the key BEFORE approaching the door.')

Strict rules prohibit: coordinates, grid positions, speculation about unseen objects, markdown formatting, and vague statements. All reflections are validated and rejected if they contain coordinates or positional language.

10.5 Reflection Memory Storage

Validated reflections are stored in a bounded circular buffer with the following properties:

- **Maximum size:** 20 entries. When full, the oldest entry is evicted (FIFO).
- **Deduplication:** Each reflection is hashed using SHA1. If an identical reflection is submitted, its frequency counter is incremented rather than creating a duplicate entry.
- **Cluster labeling:** Each reflection is assigned to a cluster (e.g., KEY_FIRST, EXPLORE_FRONTIER, DIRECT_NAVIGATION) based on its content. This enables cluster-prioritized retrieval.
- **Success tracking:** Each entry tracks how many times it came from a successful versus failed episode, enabling success-rate-based ranking.

10.6 Reflection Retrieval for Planning

When the LLM needs to be called (cache miss), the system retrieves up to 5 relevant reflections from the memory buffer. Retrieval is NOT random — it is ranked by:

- **Cluster match:** Reflections matching the current state's cluster are ranked first.
- **Success rate:** Reflections associated with successful outcomes are ranked higher.
- **Frequency:** Reflections seen more often are ranked higher.

A token budget of 500 tokens is enforced. Reflections are included one by one until the budget is exhausted. The retrieved reflections are injected into the LLM prompt before the planning question, so the LLM is aware of past strategies when generating a new plan.

10.7 Reflection Statistics from Training

Metric	Value	Meaning
Reflections generated	15,840	Total calls to the reflector LLM (Ollama backend)
Reflections validated	15,511	Passed content validation check
Reflections rejected	329	Contained coordinates or invalid content

Metric	Value	Meaning
Unique entries stored	15,418	New unique lessons added to memory
Duplicate entries merged	93	Identical reflections — frequency bumped
Reflections from successes	15,177	97.8% of stored reflections came from successful episodes
Reflections from failures	334	2.2% from failed episodes (rate-limited to every 3 episodes)
Memory retrieval events	475	Times the memory buffer was queried (= number of LLM calls)
Total reflections retrieved	2,348	Individual reflection texts injected across 475 calls (4.94 avg)
Buffer size at end	20 / 20	Buffer was full; final 20 entries are all SUCCESS cluster (FIFO eviction from high success rate late in training)
Reflector backend	Ollama	Local LLM serving (not an external API)

11. Training Charts and Analysis

The following charts show actual training metrics recorded during the Kaggle run. All charts are taken directly from the Kaggle notebook output.

11.1 Success Rate and Episode Outcomes

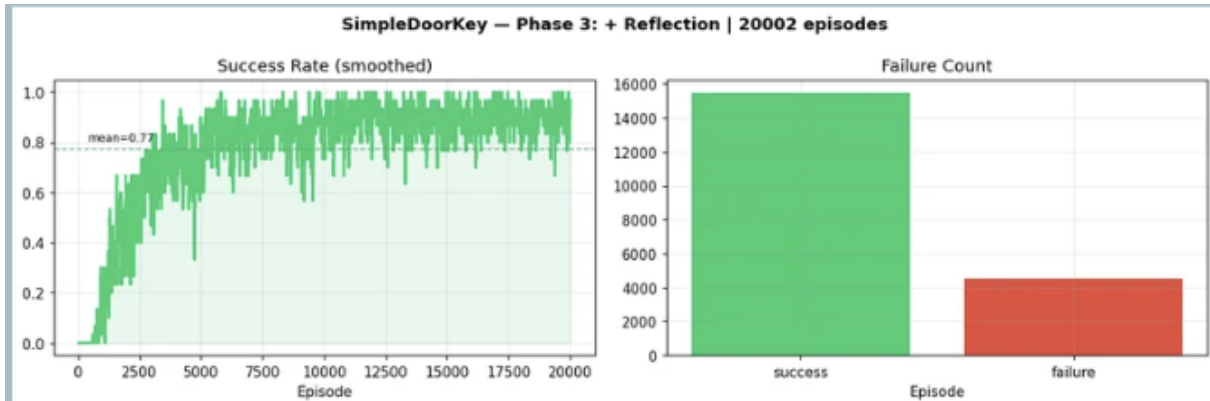


Fig. 1 — Per-episode success rate (smoothed) and total episode outcome counts. Left: Success rate rises from 0% to near 100% over 20,002 episodes. Mean = 0.78 (78%). Right: Approximately 15,482 successful episodes (green) vs 4,518 failed (red).

Overall success rate: 77.4% across 20,002 training episodes. The agent first reached 100% per-iteration success at iteration 343. The mean per-iteration success rate across all 2,000 iterations is 78%.

Failure breakdown (4,518 failed episodes): 3,274 episodes — agent acquired the key but failed to interact with the door correctly. 977 episodes — key was visible but the agent could not navigate to it. 267 episodes — the key was never found. Zero episodes opened the door without first picking up the key — the LLM plan structure was always semantically correct.

11.2 Average Return (Reward)

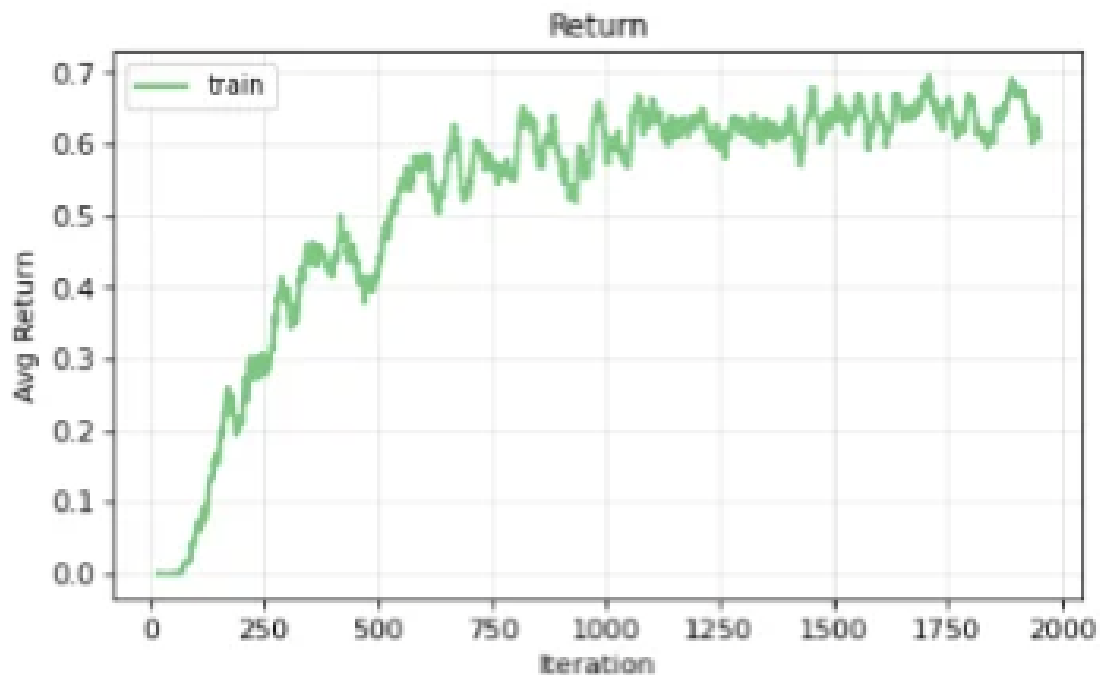


Fig. 2 — Average return per iteration over 2,000 iterations. Rises from 0.0 at the start to approximately 0.65 plateau from iteration 500 onward.

Average return rises as the agent learns to complete the task in fewer steps. A return of 0.65 corresponds to approximately: $1 - 0.9 \times (\text{steps}/150) = 0.65 \rightarrow \text{steps} \approx 58$. This means successful episodes in late training complete in around 58 steps on average.

11.3 Episode Length Decrease

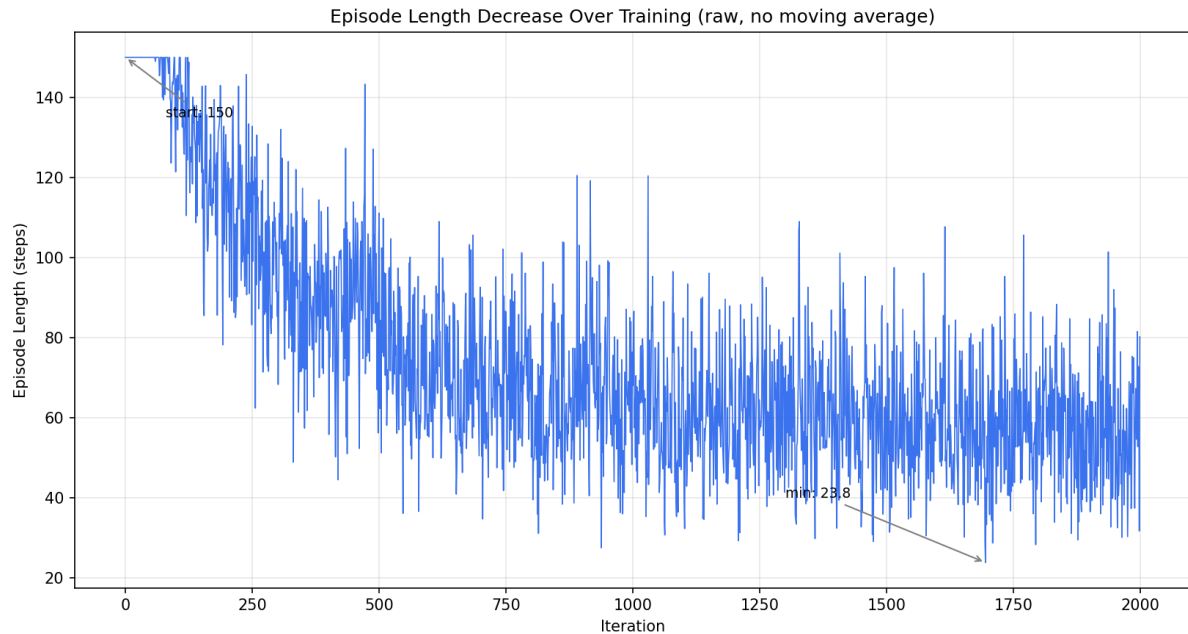


Fig. 3 — Episode length per iteration (raw, no moving average) over 2,000 iterations. Starts at 150 steps (maximum) and decreases to around 23-40 steps in late training.

The maximum episode length is 150 steps. In early training, nearly all episodes reach this limit because the agent has not yet learned to complete the task. As training progresses and the success rate rises, more episodes terminate early with success. Later in training, successful episodes typically complete in around 40-60 steps, pulling the average episode length down significantly.

11.4 Kickstarting Loss

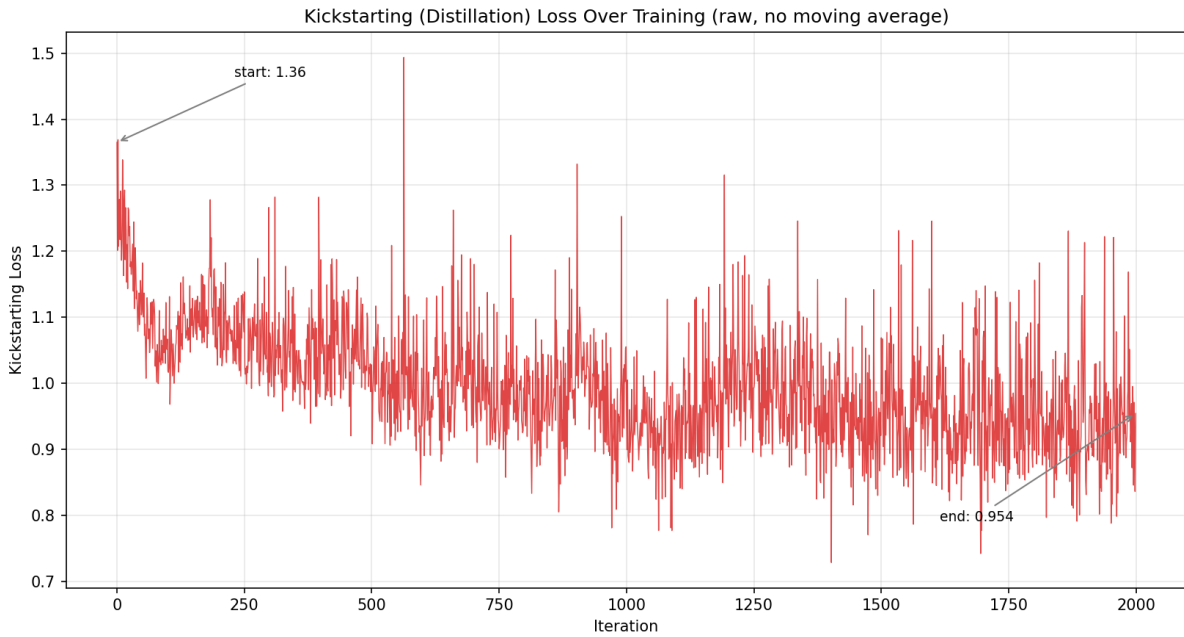


Fig. 4 — Kickstarting (distillation) loss over 2,000 iterations. Starts at 1.36, ends at 0.954. Decreasing trend throughout training.

The KS loss measures cross-entropy between the teacher's action distribution and the student policy's softmax output. It starts high because the student policy is initially random and produces a near-uniform distribution, while the teacher concentrates probability on a specific action. The loss decreases as the student learns to align its distribution with the teacher's recommendations. It does not reach zero because the teacher distributes weight across both the primary plan and the explore backup, and the student also maintains some probability on non-primary actions for exploration.

11.5 Kickstarting Coefficient Schedule

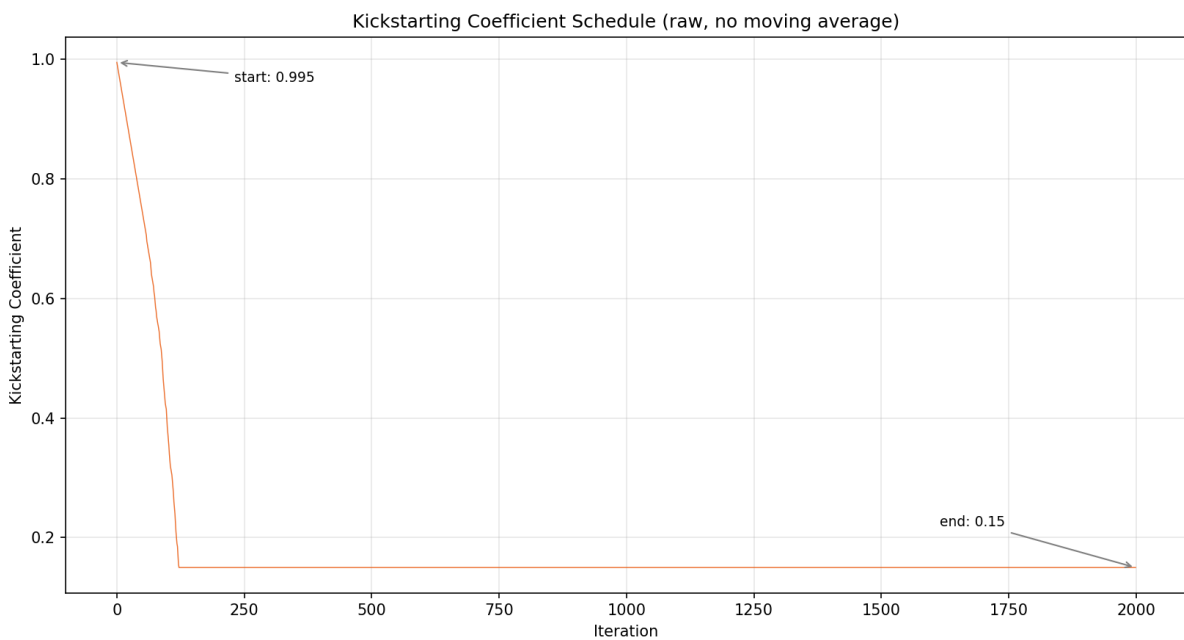


Fig. 5 — Kickstarting coefficient over 2,000 iterations. Starts at 0.995, reaches floor 0.15 at iteration 122, stays constant thereafter.

The kickstarting coefficient controls how much the teacher influences the total loss. It starts at 0.995 and decays based on the recent success rate. The teacher influence coefficient decreases during training and eventually reaches a constant floor value of 0.15. Teacher guidance therefore remains active throughout training, although its effect becomes much smaller after convergence.

Phase 1 (iterations 0-70): No successes yet. Base decay rate: 0.005 per iteration. After 70 iterations: $0.995 - (70 \times 0.005) = 0.645$.

Phase 2 (iterations 70-122): First successes appeared. With success rate reaching 10-30%, decay accelerated to 2× or 3× speed (0.010-0.015 per iteration). The floor 0.15 was reached at iteration 122.

Phase 3 (iterations 122-1999): Coefficient fixed at floor 0.15. Teacher contributes $0.15 \times L_{KS}$ to the total loss for the remaining 1,878 iterations.

11.6 Policy Entropy

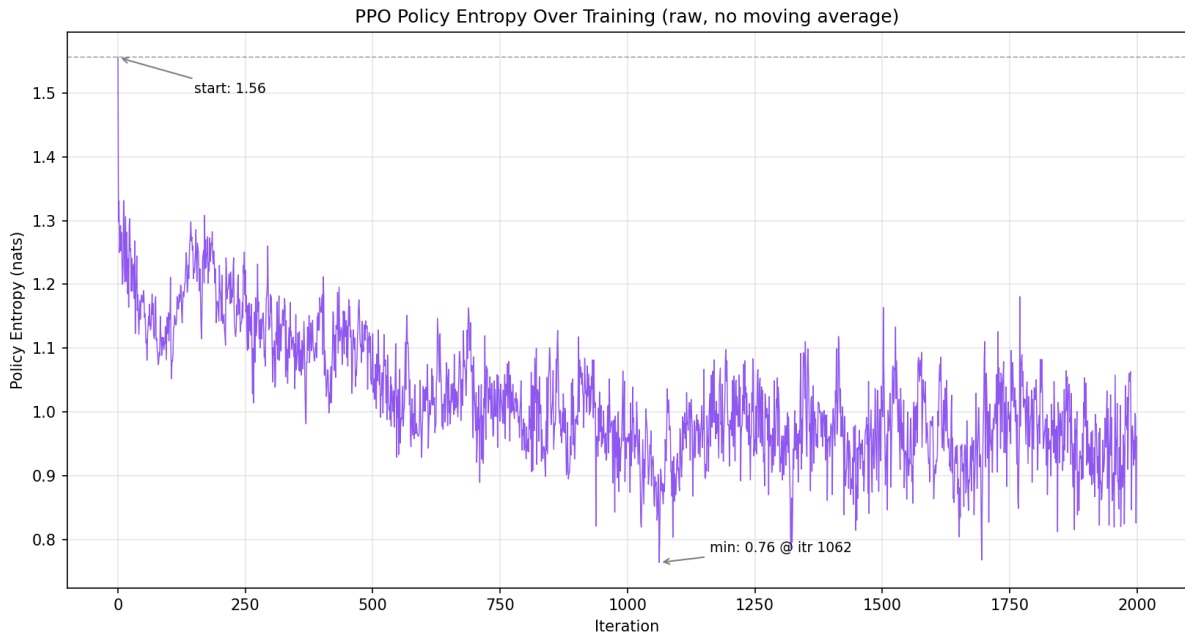


Fig. 6 — PPO policy entropy over 2,000 iterations. Starts at 1.56 nats. Minimum 0.76 nats at iteration 1,662. Ends at approximately 0.96 nats.

Policy entropy measures how spread out the agent's action distribution is. A high entropy means the agent is exploring broadly (uncertain about what to do). A low entropy means the agent has converged toward specific actions. The dynamic entropy coefficient prevents collapse: if entropy drops below 0.5 nats, the entropy bonus doubles; below 0.2 nats, it triples. The entropy ends at 0.96 nats, indicating the agent still maintains healthy exploration while having a clear directional preference.

11.7 Student-Teacher Action Agreement

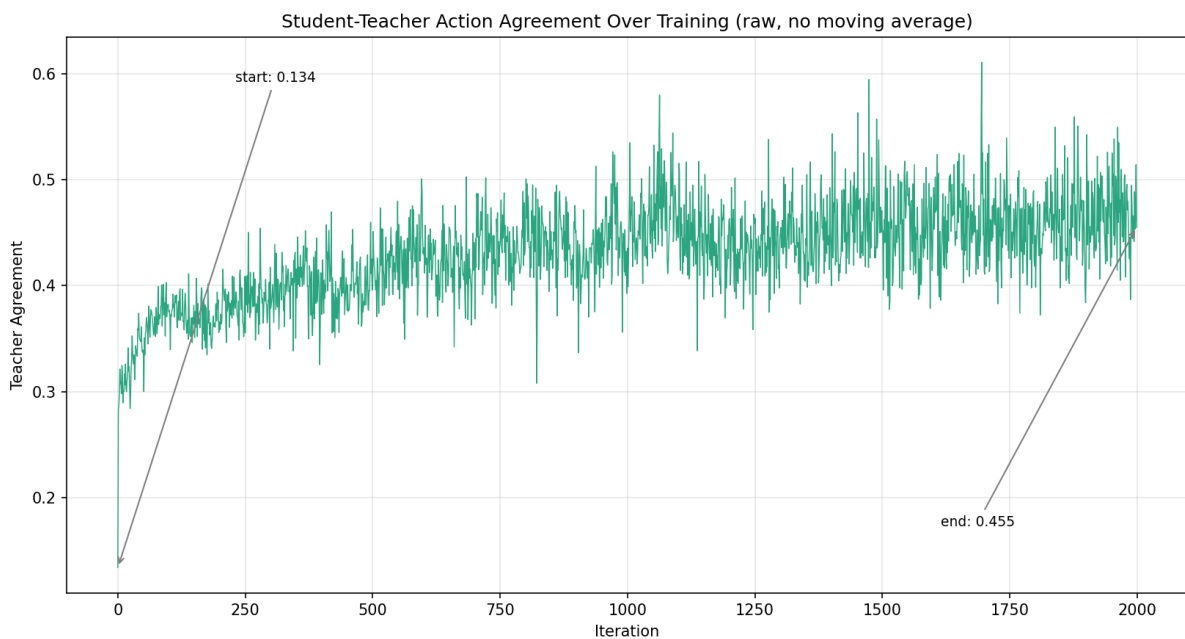


Fig. 7 — Student-teacher action agreement over 2,000 iterations. Starts at 0.134 (13.4%), rises to 0.455 (45.5%) by the end of training.

Action agreement measures the fraction of steps where the student PPO policy chooses the same action as the teacher's top recommendation. It starts low (13.4%) because the student is initially random, and rises to 45.5% as the student internalizes the teacher's navigation strategy. The agreement does not reach 100% because: (1) the teacher's distribution has a 15% weight on the explore backup action, and (2) the student maintains exploration probability across multiple actions.

12. Complete Kaggle Training Statistics

The following table contains verified statistics from the Kaggle training run. All values are sourced directly from history.json, llm_stats.json, and eval_metrics.json.

Category	Metric	Value	Notes
Training	Total iterations	2,000	Each iteration = 10 episodes + PPO update
Training	Total episodes	20,002	20,000 base + 2 buffer top-up episodes
Training	Total timesteps	1,500,657	One planner query per step
Training	Episodes per iteration	10 (base)	Plus top-up when buffer is under 256 transitions
Training	PPO epochs per iteration	8	8 full passes over the buffer each iteration
Training	Batch size	128 transitions	Mini-batch size for each PPO update
Success	Total successful episodes	~15,482	77.4% overall success rate
Success	Total failed episodes	~4,518	22.6% failure rate
Success	Mean per-iter success rate	77.4%	Mean of per-iteration success fractions
Success	First 100% success iteration	343	First iteration where all 10 episodes succeeded
LLM Cache	Total planner queries	1,500,657	One per step — includes all cache hits
LLM Cache	Cache hits	1,500,181	99.97% served from cache
LLM Cache	Actual LLM calls	476 (0.03%)	6 initial plans + 470 after cache invalidations
LLM Cache	Cache invalidations	470	Plan deleted after 5th failure on a state
LLM Cache	Unique symbolic states ever	6	Only 6 distinct RL2LLM strings in training
Failure	Total interventions fired	4,725	Stuck + oscillation + failed-interaction events
Reflection	Reflections generated (Ollama)	15,840	LLM called after episodes to reflect

Category	Metric	Value	Notes
Reflection	Reflections validated	15,511	Passed content check
Reflection	Reflections rejected	329	Contained coordinates or invalid content
Reflection	Unique lessons stored	15,418	New entries added to memory buffer
Reflection	Duplicate merges	93	Same lesson seen again — frequency bumped
Reflection	From successful episodes	15,177 (97.8%)	Successes always trigger reflection
Reflection	From failed episodes	334 (2.2%)	Failures rate-limited to every 3 episodes
Reflection	Memory retrieval events	475	Number of times memory was queried for planning
Reflection	Total texts retrieved	2,348	Avg 4.94 per call — confirms top_k=5
Reflection	Reflections injected in prompts	475	475 of 476 LLM calls included memory context
KS Coef	Initial coefficient	0.995	experiment_config.py kickstarting_coef_initial
KS Coef	Floor coefficient	0.15	experiment_config.py kickstarting_coef_minimum
KS Coef	Decay rate (base)	0.005 / iteration	experiment_config.py kickstarting_coef_descent
KS Coef	Floor reached at iteration	122	Confirmed from history.json
KS Coef	Iterations at floor	1,878 of 2,000 (94%)	Majority of training at minimum influence

13. Conclusion

LLM4Teach demonstrates that symbolic LLM planning can effectively guide a PPO agent in a sparse-reward environment — without requiring the LLM to be called at every step. The key architectural insights are:

- **77.4% training success rate** over 20,002 online episodes in the MiniGrid SimpleDoorKey environment, with the agent first achieving 100% per-iteration success at iteration 343.
- **Extreme LLM efficiency:** 476 LLM calls across 1,500,657 training steps. The 6-state plan cache achieves a 99.97% hit rate, making the LLM a one-time knowledge source rather than a real-time oracle.
- **Teacher influence remains active throughout training.** The kickstarting coefficient decays to its floor of 0.15 at iteration 122 and stays there. The teacher does not disappear — it provides a consistent 15% influence on every subsequent gradient update.
- **Kickstarting propagates LLM knowledge through every gradient step**, not just the 476 calls. The cross-entropy loss shapes the student's entire action distribution at every mini-batch update.
- **Reflection closes the feedback loop:** 15,418 unique strategic lessons were stored and 475 times the memory was queried during LLM planning calls, providing relevant past experience at every new plan generation event.
- **Self-healing behavior:** 4,725 automatic failure interventions and 470 cache invalidations kept the planning system responsive without any manual intervention.

End of Report